Final Project Description

COE891: Software Testing and Quality Assurance

Course Instructor: Dr. Reza Samavi

1 General Project Description

Software testing is an important aspect of creating reliable software. Therefore, the final project is the main component of this course. The objectives of this project are to prepare you (as a member of a software development team) to detect errors, faults and failures in a given application and plan to rectify the problems using the methods you learned in his course.

Project Composition: The project will be completed by a group of 4 to 5 students as a team. A software or web-based open-source application should be selected by each group as the project topic. The size of the selected project should be reasonable and large enough to simulate enterprise software testing which will be further described in the following sections.

Moreover, the project should be performed based on a proposed test plan by each group. Each group needs to propose a test plan containing the tools and testing methods covered by the course to be used for testing the selected Java open-source application. The selected project topic and the test plan should be submitted and approved by the course staff and/or the course instructor. The test plan should contain the general information application (including open source URL, number of classes and methods, number of units, and major software features), applicable testing methods, and required testing tools for each test method.

LLM-based Software Testing: In addition to regular software testing activities, each group will conduct a small experiment on using Large Language Models (LLMs) as testing assistants. In particular, students will interact with an open-source code LLM (such as Code Llama, StarCoder, or GPT) to generate or refine test cases for selected Java methods in their chosen application. The goal is to critically evaluate how LLM-generated tests compare to manually designed tests in terms of coverage, fault detection, and overall reliability, and to analyze/reflect on the benefits and limitations of LLM-based software testing. An example of such a code-oriented LLM is StarCoder, a 15B-parameter model trained on source code from 80+ programming languages.

A final written report of the project is also required to successfully complete the project. The final written report should contain the finalized test plan, all findings, critical analysis, barriers, and results achieved by applying different testing methods and tools on the open-source application.

1.1 Preliminary Study for Topic Selection

Before you select a topic, we recommend you studying the following sources:

- This sample project, which is similar to your project, will help you to understand the level of details in your report, the discussion, critical analysis, and expectations for the project deliverable. The groups may need to combine two or more similar submission contents into one single submission as described in the Project Deliverable section of this document.
- How software companies test their software?
- Java test automation frameworks
- Performance testing frameworks

1.2 Resources and Testing Tools

The following testing tools and resources can help you develop your project testing strategies:

- JUnit 4: A unit testing tool.
- Selenium: A testing tool for web-based application and browser testing.
- JMock: A testing tool for test-driven development in Java to test the interactions between the objects.
- JMeter: The Apache JMeter application is an open source software designed to load test functional behavior and measure performance. It was originally designed for testing web applications but has been expanded to other test functions.
- TestNG: A unit testing tool inspired from JUnit and NUnit but introducing some new functionalities that make it more powerful and easier to use.
- Mockito: A unit testing tool that lets you write elegant tests with a clean and simple API.
- PowerMock: A unit testing tool with more powerful capabilities.
- μJava: muJava (μJava) is a mutation system for Java programs. It automatically generates mutants for both traditional mutation testing and class-level mutation testing. It can test individual classes and packages of multiple classes.
- LLM-based Testing Assistants: Open-source large language models specialized for code, such as StarCoder or Code Llama. These models can generate unit tests, suggest input partitions, or provide explanations about code under test. Students are expected to use at least one such model in their project in an automated manner (i.e., not directly prompting through LLMs) and critically evaluate its outputs using the testing techniques introduced in this course.
- Introductory material on using LLMs in software engineering and software testing, for example:
  - StarCoder models and documentation
  - StarCoder blog post
  - Code Llama overview
  - A recent survey[1] on LLM-based test generation and automated software testing titled "A Review of Large Language Models for Automated Test Case Generation"
  - A recent paper[2] on LLM-based integration of mutation testing into unit test generation
  - Public benchmarks and datasets for testing LLMs on test-case generation tasks, e.g., the TestEval benchmark

1.3 Sample Project Topics (Open Source Applications)

- JBrick: An open source application which is an accessible Lego Mindstorms NXT programming environment for students and hobbyists.
- TuxGuitar: An open source application which is a multitrack guitar tablature editor and player written in Java, and it can open GuitarPro, PowerTab and TablEdit files.
- NotePad++: An open source software which is a free source code editor and Notepad replacement that supports several languages.

[1] Celik, A., & Mahmoud, Q. H. (2025). A Review of Large Language Models for Automated Test Case Generation. Machine Learning and Knowledge Extraction, 7(3), 97.

[2] Kovtko, A., Savkiv, V., Kozbur, H., Kozbur, I., & Trembach, R. (2025). Integration of Mutation Testing into Unit Test Generation Using Large Language Models.

2 Forming a Team/Group

Software testing in the industry is a team activity. In order to resemble real-world testing projects, the final project of this course is designed as a teamwork activity and cannot be completed individually. A valid team has at least 4 and at most 5 students. The students will select their teammates before the assigned deadline. After self-enrollment deadline, unenrolled students will be randomly assigned to a team by the course instructor.

Note: Since many project activities occur during lab hours, all members of a team should be from the same lab section - no exception will be accommodated.

3 Project Topic Selection

Each team should select a distinct project (open source application) of a software application or a web-based application written in Java. The teams should clearly define the objectives and more importantly the scope of their selected open source project. The topic of the project is of the team's own choice based on the predefined rules described as follows:

- Any of the two groups should not have the same project topic.
- If two teams accidentally select the same open-source application for a testing project, the selected open-source application will be assigned to the team that selects the application first at the discretion of the course staff.
- The source code of the selected open-source applications should be written in Java or Python. However, if any of the teams wish to use an application written in another programming language, they will need to discuss and justify their selected case with the course instructor prior to starting their project.
- The selected projects should be a medium-sized open source applications with at least 100 classes. The chosen projects should be large enough to cover several test cases and simulate an enterprise software testing environment.
- Projects with a high level of documentation are preferred.
- The selected project must be approved by the course TA and/or instructor.
- **Submission Instructions:** Your group submission (to the group project folder under the Assignments folder on D2L) will be a single PDF file titled as SectionXX_GroupYY.pdf containing the following information:
  - Project Title (the topic of an open-source Java application).
  - The links to the project repository/implementation/codes in Github or other repositories.
  - A short abstract as your project description in less than 200 words.
  - Information of your project group members including, Full Name, Student ID, and TMU Email address.

4 Project Test Plan

Each team should provide a test plan regarding the tools and testing methods that are going to be used in their software testing for each part of the selected open-source application and meet the following requirements:

- The test plan should contain the application's general information, application architecture (classes and main methods information) and features, applicable testing methods (e.g., Input Space Partitioning, Graph Coverage, Logical coverage, Syntax Coverage and Mutation Testing) and required testing tools for each specific part.
- The groups are provided with a template of the desired architecture of a test plan which is described and illustrated in the `Test_Plan_Template.pdf` file attached to this project file.

- Note that test plans from different groups are expected to be different and must reflect the nature of selected open-source code.
- Groups should try to provide their best possible test plan which is compatible with their selected projects. However, the teams do not necessarily need to adhere to their provided test plans because of unforeseen but expected barriers in designing and developing test requirements and test cases. In addition, the students will learn new concepts in the lectures as they make progress in analyzing their source code. Therefore, the groups are allowed to include new test methods not known at the time of writing the test plan.
- Note that the progress checkpoint includes a Q&A part in which the team members should answer the TA questions. All the group members must be present in progress checkpoint.

5 Project Tasks and Deliverables

The group members should meet every week after topic selection to discuss their project progress during their lab session and revise their test plans based on feedback provided by the TAs. In addition, there will be an official project progress checkpoint in Week 9 (Lab IV) where the groups can ask their questions and discuss their concerns with the TAs.

5.1 Testing Abstractions/Approaches

Groups should **apply each one of the following testing abstractions/approaches to 10 different methods in at least 5 different Java classes or at a similar complexity level in Python** along with incorporating **LLM-assisted testing component** detailed in Section A. For each test, you will determine test values and use Unit Testing. Based on your project topic and application, you may use standard JUnit, JUnit Parameterized, JUnit Theories, and TestNG. Obviously, you will **implement/code** your testing programs.

You will need to cover 4 different testing abstractions out of the following 5 test methods in your final project:

1. **Input Space Partitioning (ISP):** You will need to use Boundary Value Analysis (BVA) to test your input space based on the application of a Java class. Obviously, you will implement/code your testing programs.
2. **Graph-based testing (CFG and DFG):** You will need to draw CFGs and provide the corresponding analysis based on different coverage methods for control flow. You will need to draw DFGs and provide the corresponding analysis based on different coverage methods for data flow.
3. **Logic-based Testing:** You will need to provide the logical analysis based on different coverage methods for logical predicates/clauses.
4. **Mutation Testing:** You will need to modify the program under test in small ways, e.g., flipping a less-than sign to a greater-than or changing a hard-coded number from 0 to 1. These are usually designed to mimic typical programming errors, such as typographical errors, wrong choice of operator, or off-by-one errors.
5. **Formal Methods:** You will need to select a portion of the program (e.g., specific safety-critical methods), provided with justifications, to conduct verification methods and prove the program or a specific method is behaving as expected. The selected method should be complex enough to cover different aspects of formal verification.

5.2 LLM-assisted Testing Component

In addition to the regular testing activities described above in Section 5.1, each group must complete an LLM-assisted testing component fully described in Appendix A and B. The purpose of this component is to help students become familiar with large language models for code development and to critically evaluate how LLMs can and cannot support software testing in practice.

Selection of Methods

Each group selects a subset of their program under test for the LLM experiment:

- Choose **at least** 3 **and at most** 5 **Java methods** from your selected open-source application.
- These methods must also **be part of the set of at least** 15 **methods** to which you apply Input Space Partitioning (ISP), graph-based testing, logic-based testing, mutation testing, and formal methods.
- The selected methods should have non-trivial control flow and/or logical conditions (e.g., multiple branches or predicates), so that LLM-generated tests can be meaningfully evaluated.

Manual Test Design

For each selected class and method under test, students must first design a *manual* test suite using the techniques from the rest of the project. In particular, for each chosen method:

- Define test requirements using input space partitioning (ISP), including appropriate partitions and boundary values.
- Derive concrete test inputs to satisfy the relevant control-flow and data-flow coverage criteria (CFG/DFG) introduced in the course, where applicable.
- Apply logic-based coverage criteria to predicates and clauses (e.g., condition/decision coverage), when meaningful.
- Implement the resulting test cases as JUnit tests under `src/test/java` using JUnit 5.

This manually designed test suite will serve as the *baseline* for comparison with the LLM-generated tests produced by the ChatUniTest framework.

Using an LLM as a Testing Assistant via ChatUniTest

After constructing the manual test suite, each group must use the provided ChatUniTest framework, described in Section A, to generate additional tests with a code-oriented LLM (e.g., `codeqwen:v1.5-chat` configured via a local OpenAI-compatible endpoint). You do not need to interact with the LLM directly; instead, you will drive it through Maven using ChatUniTest.

Conceptually, ChatUniTest constructs prompts that include the class under test (signatures, method bodies, and surrounding context) and asks the LLM to propose and refine JUnit test cases. You are not required to hand-craft prompts, but you will understand that the LLM is being used to generate candidate tests based on the code and its intended behavior.

Evaluation and Comparison

You must evaluate the LLM-generated tests using the same techniques and tools that you apply to your manually designed tests. At a minimum, for each selected class, you should:

- **Structural coverage.** Measure statement and branch coverage for:
  1. the manually written test suite (manual tests only);
  2. the LLM-generated test suite (tests produced by ChatUniTest only);
  3. the combination of both (manual + LLM-generated tests).

  Use the coverage tools already employed in the rest of the project.
- **Mutation-based evaluation.** Run the existing mutation testing setup included with the project (e.g., PIT or the configured mutation tool) to compute the mutation score for:
  1. the manual test suite, and
  2. the combined manual + LLM-generated test suite.

If the automated mutation tool cannot be executed in your environment, you must perform a manual mutant experiment by introducing small controlled changes in the code (e.g., modifying a comparison or removing an exception check) and observing which tests (manual vs. LLM-generated) detect the mutant.

- **Qualitative analysis of LLM-generated tests.** Identify and classify issues and patterns in the ChatUniTest-generated tests, such as:
  - non-compiling tests or incorrect API usage (before your fixes),
  - incorrect assertions or unrealistic expected values,
  - redundant or equivalent tests (tests that do not increase coverage or mutation score),
  - missing tests for important boundary cases, branches, or exceptional behavior.

Based on these observations, you should analyze whether and how the LLM-generated tests help kill additional mutants or increase coverage, and discuss possible reasons (e.g., better exploration of corner cases, or conversely, systematic LLM mistakes).

Reporting and Critical Reflection

In your final report, you must clearly document how you used the LLM and ChatUniTest and how you evaluated the resulting tests. At a minimum, include:

- A brief description of the chosen LLM configuration and framework:
  - the model name (e.g., `codeqwen:v1.5-chat`),
  - how it was accessed (e.g., via a local OpenAI-compatible endpoint configured in ChatUniTest),
  - and the relevant ChatUniTest/Maven commands you used.
- A concise description of the information given to the LLM by ChatUniTest (e.g., class under test, method signatures, context), and a short explanation of how MUTAP differs from the default scheme, if you used it.
- A quantitative comparison of manual vs. LLM-assisted tests, reporting for each selected class:
  - statement and branch coverage for (1) manual tests, (2) LLM-generated tests, and (iii) combined tests;
  - mutation scores (or results of the manual mutant experiment) for manual vs. combined test suites;
  - any additional metrics requested in the project (e.g., number of failing tests detected on seeded faults).
- A qualitative analysis discussing:
  - in what ways the LLM-generated tests were helpful or efficient (e.g., speed of generation, variety of inputs, discovery of unexpected corner cases),
  - common failure modes or limitations of the LLM-generated tests (e.g., incorrect assertions, poor treatment of exceptional behavior, missing boundary conditions, redundant tests),
  - how LLM-assisted test generation complemented (or failed to complement) your manual test design,
  - and whether you would consider using such tools in a real testing workflow, and under what conditions (e.g., always with human review, in combination with mutation testing).

Your analysis should demonstrate that you have applied the testing concepts learned in this course (partitioning, structural coverage, logic-based criteria, mutation testing) to critically evaluate the LLM-assisted tests. The goal is *not* to treat the LLM's output as automatically correct, but to understand when and how such tools can be useful in a principled software testing process.

6 Submission Instructions

The **expectations and important points** regarding your project and submission are as follows:

- Each group may distribute the testing tasks among the group members, in terms of testing approaches or number of methods/classes, to mitigate the workload and make equal contributions.
- Each group only include 1-2 examples for each one of the applied testing approaches in their final report.
- Each group will submit/upload the final written report as a single PDF file, and the entire accomplished testing tasks including your implementations/codes, graphs, screenshots, and coverages/analysis as a single ZIP file, to your group project folder in D2L.
- After your corresponding lab session in Week 13 passed, a 10 percent penalty for each day of your late final written report submission will be applied. Please note you are not allowed to upload files greater than 10MB. If your project size is larger, it means you're including dependent libraries in to your project. Instead, you have to include only project files that you have been working on and then include a `requirements.txt` file in the root directory of the project. The file contains all packages installed in the project and their corresponding version.
- The **final project report** should include (but not be limited to) the following items:
  - Findings and results of the application testing
  - The informative list of test requirements, test cases, testing methods, and testing tools as well as the snapshots of testing tools results, diagrams and graphs for coverage and testing process attached by informative and concise description and justification for each specific part of the application
  - A dedicated subsection on the LLM-assisted testing component, including:
    * a description of the selected methods and corresponding prompts,
    * representative LLM-generated tests (with any corrections you had to make),
    * coverage and mutation results comparing manual vs LLM-assisted tests,
    * your critical reflection on the effectiveness and limitations of LLM-based software testing in the context of your project.
- The groups should look for some types of bugs, faults, and errors, and test their own application. These types of faults are (but are not limited to) the following:
  - Minor or Non-functional
  - Incorrect Documentation
  - Incorrect Functionality, like classes, methods, or variables functionality issues
  - Crashing.
  - Poor Performance, particularly in web-based applications
  - Security Issues, particularly in web-based applications like login issues, etc.
- Moreover, in the final report, groups should reflect on what went well and what went poorly, and how they could improve each part of the project. In addition, the final report should include how each group decided to choose their testing tools and approaches.
- Furthermore, the critical analysis should be included in the final report for all potential difficulties and barriers the groups encounter.
- Evaluations and grading are performed based on how extensive and comprehensive the tests are, how well the groups present their project and test plan, the quality of the written reports, how well the groups exploit the testing tools and techniques to fulfill testing process, and how well the groups answer and justify the project questions.

You have to include the entire history of what queries you have asked an LLM and what answer the LLM reported back (the entire Q&A you had with the LLM) for fixing the potential errors, issues, etc. and include this conversation for each question separately in the Appendix to your assignment. If you fail to do so and the use of LLM is discovered for any part of the assignment, you will get a zero for the entire assignment, and the submitted work will be treated as plagiarism. If you are interviewed about the assignment, you should be able to rationalize the answers provided in the assignment and do the assignment independently of using any LLM if being asked; otherwise, the use of LLM will be considered plagiarism.

7 Scheduling and Marking Scheme

The course project is worth 10% of the course grade. Further breakdown of the project grade is as follows:

- **Week 2:** Team Formation
- **Week 4:** Project Topic Selection
- **Week 9:** Project Progress Checkpoint: Test Plan Submission (20% of the project grade)
- **Week 13:** Final Written Report Submission with appendices as a single PDF file (80% of the project grade)

Note: While the project is teamwork, the grade will be assigned to each individual. The assumption is that all members of the team are equally contributing to the course project. In case there is a conflict between team members or the course staff/instructor realizes the contributions are not uniformly distributed among the team members, the course instructor will have the full discretion to assign the project grade to each individual.

A LLM-assisted Testing: ChatUnitest Maven Plugin

Prior work has explored using large language models (LLMs) to support a range of programming tasks, often with promising outcomes. However, using an LLM in a direct-prompting workflow introduces practical constraints. First, generated code frequently fails to run as-is, echoing the common experience of "five minutes of coding, two hours of debugging." Second, incorporating model outputs into an existing codebase can be inconvenient, since it typically requires manual, out-of-band interaction and repeated context switching across tools and pages. To mitigate these issues, the **ChatUniTest** framework[3] proposes a Generate-Validate-Fix workflow and implements a prototype system, along with plugins designed to integrate smoothly into established development pipelines. The Maven plugin is now published to the Maven Central Repository for your trial and feedback. Additionally, the Chatunitest plugin has been launched in the IntelliJ IDEA Plugin Marketplace. You can search for and install ChatUniTest in the marketplace or visit the plugin page Chatunitest: IntelliJ IDEA Plugin for more information. You can check out the ChatUniTest repository for more details on the properties and configuration parameters.

You will run an open-source code model locally using a tool such as `Ollama` or `LM Studio`, which exposes an OpenAI-compatible HTTP endpoint on the local machine. In this case, the ChatUniTest plugin is configured to call the local endpoint (e.g., `http://localhost:11434/v1/chat/completions`) with a dummy API key, and no external paid API is required.

An Example: LLM-assisted Test Generation in Eclipse Using ChatUniTest and a Local LLM

This section provides a complete, practical example that starts from an empty Maven project in Eclipse and ends with running LLM-generated unit tests. The setup uses:

- Eclipse as the IDE,
- Maven as the build tool,
- ChatUniTest as the LLM-based test generation plugin, and
- a local code-oriented LLM (CodeQwen v1.5 chat) served by Ollama on `localhost`.

Prerequisites. Students should have the following installed:

- Java JDK (version 11 or later),
- Maven,
- Eclipse (any recent version with Maven support),
- Ollama (for running a local LLM).

Step 1: Local LLM Setup (Ollama + CodeQwen)

1. Install Ollama from `https://ollama.com/` and ensure it is running (on Windows/macOS, start the Ollama app; on Linux, you may run `ollama serve`).
2. Open a terminal or command prompt and pull a code model that is supported by both Ollama and ChatUniTest, e.g.:

   ```
   ollama pull codeqwen:v1.5-chat
   ```

   You can check the list of installed models by running

   ```
   ollama list
   ```

   Then, you should see a line with `codeqwen:v1.5-chat`.
3. Test the model locally using the OpenAI-compatible chat endpoint exposed by Ollama (typically `http://localhost:11434/v1/chat/completions`). For example:

[3] Chen, Y., Hu, Z., Zhi, C., Han, J., Deng, S., & Yin, J. (2024, July). Chatunitest: A framework for LLM-based test generation. In Companion Proceedings of the 32nd ACM International Conference on the Foundations of Software Engineering (pp. 572-576).

```
curl -X POST http://localhost:11434/v1/chat/completions ^
  -H "Content-Type: application/json" ^
  -H "Authorization: Bearer ollama" ^
  -d "{\"model\": \"codeqwen:v1.5-chat\", \
       \"messages\": [{\"role\": \"user\", \
       \"content\": \"Say this is a test.\"}]}"
```

If a JSON response is returned with an assistant message, the local LLM endpoint is working correctly.

Step 2: Create a Maven Project and Class Under Test in Eclipse

Here, the example is based on the class `Calculator`. You may replace `Calculator` with any method/class from your chosen project and follow the same steps.

2.1 Create a new Maven project.

1. In Eclipse, select *File* → *New* → *Maven Project*.
2. Accept the default workspace location and click *Next*.
3. In the archetype selection dialog, filter for `quickstart` and select:

   `maven-archetype-quickstart`

   then click *Next*.
4. Provide the project coordinates (these values are used in this example):
   - Group Id: `com.example`
   - Artifact Id: `calculator-chatunitest`
   - Version: default
   - Package: `com.example`

   Click *Finish*. Eclipse creates a new Maven project with `pom.xml`, `src/main/java`, and `src/test/java`.

2.2 Create the class under test.

1. In the Package Explorer, expand `calculator-chatunitest` → `src/main/java` → `com.example`.
2. Delete any template class (e.g., `App.java`), if created.
3. Right-click the package `com.example` and select *New* → *Class*. Use:
   - Name: `Calculator`
   - Package: `com.example`

   Click *Finish*.
4. Replace the contents of `Calculator.java` with:

```
package com.example;
public class Calculator {
    public int add(int a, int b) {
        return a + b;
    }
    public int divide(int a, int b) {
        if (b == 0) {
            throw new IllegalArgumentException("b must not be zero");
        }
        return a / b;
    }
    public int max(int a, int b) {
```

```
            return (a >= b) ? a : b;
        }
    }
```

Then, save the file.

Step 3: Configure Maven Dependencies and ChatUniTest Plugin

3.1 JUnit and ChatUniTest dependencies. Open `pom.xml` in the XML editor and ensure it contains entries similar to the following (make sure that you are using the correct class name, etc.):

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
                             http://maven.apache.org/maven-v4_0_0.xsd">
  <modelVersion>4.0.0</modelVersion>

  <groupId>com.example</groupId>
  <artifactId>calculator-chatunitest</artifactId>
  <version>1.0-SNAPSHOT</version>
  <packaging>jar</packaging>

  <name>calculator-chatunitest</name>

  <properties>
    <maven.compiler.source>11</maven.compiler.source>
    <maven.compiler.target>11</maven.compiler.target>
    <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
  </properties>

  <dependencies>
    <!-- JUnit 5 -->
    <dependency>
      <groupId>org.junit.jupiter</groupId>
      <artifactId>junit-jupiter</artifactId>
      <version>5.9.2</version>
      <scope>test</scope>
    </dependency>

    <!-- ChatUniTest starter -->
    <dependency>
      <groupId>io.github.ZJU-ACES-ISE</groupId>
      <artifactId>chatunitest-starter</artifactId>
      <version>1.4.0</version>
      <type>pom</type>
    </dependency>
  </dependencies>
```

3.2 Configure the ChatUniTest Maven plugin. Within the `<build><plugins>` section, add:

```
  <build>
    <plugins>
      <!-- Surefire: run JUnit 5 tests -->
      <plugin>
        <groupId>org.apache.maven.plugins</groupId>
        <artifactId>maven-surefire-plugin</artifactId>
        <version>3.1.2</version>
        <configuration>
            <useModulePath>false</useModulePath>
```

```
            <includes>
                <include>**/*Test.java</include>
                <include>**/*Tests.java</include>
                <include>**/*TestCase.java</include>
                <!--  this line is important for mvn test -->
                <include>**/*Suite.java</include>
            </includes>
        </configuration>
      </plugin>

      <!-- ChatUniTest Maven plugin -->
      <plugin>
        <groupId>io.github.zju-aces-ise</groupId>
        <artifactId>chatunitest-maven-plugin</artifactId>
        <version>2.1.1</version> <!-- or latest -->
        <configuration>
          <!-- Dummy key: required by plugin but ignored by Ollama -->
          <apiKeys>dummy-key</apiKeys>
          <!-- Model name supported by both ChatUniTest and Ollama -->
          <model>codeqwen:v1.5-chat</model>
          <!-- Local OpenAI-compatible endpoint from Ollama -->
          <url>http://localhost:11434/v1/chat/completions</url>
          <!-- Optional: number of tests per method (default is 5) -->
          <!-- <testNumber>6</testNumber> -->
        </configuration>
      </plugin>
    </plugins>
  </build>
</project>
```

Save `pom.xml`. Eclipse should automatically download the required dependencies. Note that in your `pom.xml`, make sure to check/change the `<model>` to be one of the supported names you are using.

Step 4: Build the Project JAR File via Maven

ChatUniTest expects the compiled project JAR to exist in the `target` directory (e.g., `target/calculator-chatunitest-1.0-SNAPSHOT.jar`). If it is missing, the plugin will fail with an error such as: *"JAR does not exist. Run mvn install first."*

From the project root (the directory containing `pom.xml`), run:

```
mvn clean install
```

This command:

- cleans the `target` directory,
- compiles the main and test code,
- runs the tests, and
- produces `target/calculator-chatunitest-1.0-SNAPSHOT.jar`.

After this step, students should verify that the JAR file exists in the `target` directory.

*** Where to run Maven commands ***

All Maven commands used in this manual (e.g., `mvn clean install`, `mvn chatunitest:class`, `mvn test`) must be executed from the *project root* directory, i.e., the directory that contains the `pom.xml` file of the project. There are two common ways to do this:

- **From the command line (terminal/command prompt).**
    1. Open a terminal (on Windows: `cmd` or PowerShell; on macOS/Linux: a shell).

2. Change directory to the project root. For example:

   ```
   cd C:\Users\<your-username>\eclipse-workspace\calculator-chatunitest
   ```

3. Run the desired Maven command, e.g.:

   ```
   mvn clean install
   ```

   or

   ```
   mvn chatunitest:class -DselectClass=com.example.Calculator
   ```

   or

   ```
   mvn test
   ```

   In all cases, the working directory must be the folder that contains `pom.xml`.

- **From Eclipse using "Run As → Maven build..."**

  1. Right-click the project in the *Package Explorer* (e.g., `calculator-chatunitest`).
  2. Select *Run As → Maven build...*.
  3. In the dialog, ensure that the *Base directory* field points to the project root (the folder with `pom.xml`). By default, it is fixed and no need to change.
  4. Enter the desired command, without `mvn`, in the *Goals* field, for example:

     ```
     clean install
     ```

     or

     ```
     chatunitest:class -DselectClass=com.example.Calculator
     ```

  5. (Optional) Give it an appropriate name like `clean-install`. This name will be used to determine the configuration type in the list of different Maven configurations.
  6. Click *Run*. Eclipse will execute the Maven goal as if it had been typed in a terminal in the project root.

Running these commands from any other directory will result in errors because Maven will not be able to locate `pom.xml` and the associated project configuration.

The `test` command triggers Maven's *test* phase, which compiles the production code (`src/main/java`) and all test classes (`src/test/java`), and then executes the tests using the configured test framework (JUnit). The console output reports the number of tests run, failures, errors, and skipped tests. This is equivalent to running all JUnit tests from the IDE, but provides a reproducible, command-line based execution that can be used by coverage and mutation testing tools.

Step 5: Generate Tests with ChatUniTest

Once the JAR exists, invoke ChatUniTest *without* the `clean` phase. For example:

```
mvn chatunitest:class -DselectClass=com.example.Calculator
```

Important:

- Do not run `mvn clean chatunitest:class ...` in a single command, because `clean` deletes the JAR that ChatUniTest requires.
- The above command causes ChatUniTest to:
  1. analyze `Calculator` and its context,
  2. build prompts including the focal class and methods,
  3. call the local LLM (CodeQwen) via the configured URL,
  4. iteratively generate and repair JUnit test cases.

In the Console, you will see ChatUniTest is parsing the project, building prompts, calling the local LLM through http://localhost:11434/v1/chat/completions, and performing the "Generate-Validate-Fix" loop to ensure tests compile.

If the generation is successful, Maven will end with `BUILD SUCCESS` and a folder named `chatunitest` will be created in the project root, containing the generated test classes. Note that you may need to right-click the project, and Refresh (F5). Expand it; there will be Java test files in packages like com.example, with names (similarly) such as `CalculatorTest_ChatUniTest.java`, etc. Open one to see the structure of the JUnit tests and the inputs and assertions the LLM chose.

Note that if `target/calculator-chatunitest-1.0-SNAPSHOT.jar` does not exist and your build is failed, it means `clean install` has not succeeded yet. And if `clean install` finish with `BUILD FAILURE`, it means the earlier build error needs fixing first (missing dependency, compile error, etc.).

Step 6: Copy and Integrate Generated Tests

Copy tests into `src/test/java`. To integrate the generated tests into the standard test source tree, run:

```
mvn chatunitest:copy
```

This command copies the generated test files from the `chatunitest` directory into `src/test/java`, preserving the package structure (e.g., `com.example`). Refresh the project again; under `src/test/java/com/example`, you should now see the generated test classes.

Step 7. Run all tests with Maven or Eclipse. After copying, students can run all tests via Maven using:

```
mvn test
```

or via Eclipse:

- In Package Explorer, expand `src/test/java` and then `com.example`, right-click the project or test package or the generated each test/suite (having `Test` or `Suite`) file and select *Run As → JUnit Test*.
- **Note that you can run each one of the existing test files in `src/test/java` (copied LLM-generated tests or manually created tests) separately and manually as desired using Run As → JUnit Test.**

Then, JUnit view showing tests passing or failing. If some fail, either:

- the LLM wrote a wrong test/assertions **which must be discussed in your submission**, or
- your program/method has a bug.

Note that after any changes or fixing the issues, you may need to follow the steps again from Step 4 to make sure that you have generated the required jar file.

Step 8. Using ChatUniTest with mutation-guided generation (MUTAP). The ChatUniTest Maven plugin supports several LLM-based test generation schemes via the `phaseType` parameter, including a mutation-guided scheme based on MuTAP (Mutation Test case generation using Augmented Prompt)[4]. In this scheme, mutation testing is used inside the generation loop to identify surviving mutants and augment the LLM prompt with information about those mutants, so that subsequent test-generation rounds focus on weaknesses in the current test suite.

From the student's perspective, enabling the MUTAP scheme only requires adding `-DphaseType=MUTAP` to the usual ChatUniTest command. For an arbitrary class `fully.qualified.ClassName`, the steps are:

[4]Dakhel, A. M., Nikanjam, A., Majdinasab, V., Khomh, F., & Desmarais, M. C. (2024). Effective test generation using pre-trained large language models and mutation testing. Information and Software Technology, 171, 107468.

1. Run the existing mutation testing setup (e.g., PIT) on the manually written tests and record the baseline mutation score.
2. Invoke ChatUniTest for a selected class using the MuTAP scheme:
3. Ensure the project builds and the JAR is available:

   ```
   mvn clean install
   ```

4. Invoke ChatUniTest in MUTAP mode to generate tests:

   ```
   mvn chatunitest:class -DselectClass=fully.qualified.ClassName -DphaseType=MUTAP
   ```

5. Copy the generated tests into `src/test/java`:

   ```
   mvn chatunitest:copy
   ```

6. Run the full test suite (LLM-generated test suite) as usual:

   ```
   mvn test
   ```

The resulting tests are ordinary JUnit tests that have been generated under a mutation-guided scheme. To evaluate their effectiveness, students should compare the mutation score of:

- the manually written tests alone, and
- the combined manual + MUTAP-generated tests,

using the mutation testing tool configured for the project (e.g., PIT). Students should analyze whether MUTAP-generated tests help kill additional mutants and how this relates to the input-space, logic-based, and graph-based test design performed earlier in the project.

B Important Points, Common Issues, Questions, and Fixes

B.1 Java version issues (e.g., for lambda expressions compatibility)

The LLM-generated tests may use Java 8 features such as lambda expressions (e.g. `() -> { ...}`). If Eclipse reports an error such as:

> "Lambda expressions are allowed only at source level 1.8 or above,"

students should ensure that the project is configured for Java 8 or higher:

1. Right-click the project in Eclipse and select *Properties*.
2. Navigate to *Java Compiler*.
3. Enable *"Project specific settings"* and set the *Compiler compliance level* to `11` (or at least `1.8`).
4. Click *Apply and Close*. Eclipse will rebuild the project.

B.2 Incorrect casts from `Object` to primitive types

Occasionally, generated tests may contain minor type errors, such as incorrect casts of the form:

```
int x = (int) someObject;
```

which cause compilation errors such as:

> "Cannot cast from Object to int."

In such cases, students should cast to the corresponding wrapper type instead, letting Java auto-unbox to the primitive:

```
int x = (Integer) someObject;
```

This manual correction is part of the exercise and illustrates that LLM-generated code must still be reviewed and tidied up.

After completing these steps, you will have:

- a working Java project in Eclipse,
- a local code-oriented LLM running via Ollama,
- LLM-generated JUnit tests for the `Calculator` class,
- and the ability to execute and analyze those tests using standard tools (e.g., JUnit and mutation testing frameworks).

In the context of this course project, students should treat the LLM-generated tests as an additional test suite and evaluate them using the same criteria as their manually designed tests (e.g., statement and branch coverage, mutation score, and fault detection capability). These LLM-generated tests should then be evaluated alongside manually designed tests as described in this project manual.

B.3 Finding the local LLM URL

When using a local runner such as Ollama or LM Studio, the OpenAI-compatible endpoint is determined by that tool:

- For Ollama, the default chat-completions endpoint is `http://localhost:11434/v1/chat/completions`.
- For LM Studio, the server panel displays the base URL (e.g., `http://localhost:1234`); students should append `/v1/chat/completions` to obtain the full endpoint.

This URL is then placed in the `<url>` field of the ChatUniTest Maven plugin configuration.

B.4 Choosing an LLM and configuration

In practice, there are several choices for the underlying LLM and how it is accessed:

- **Model family and size:** In this course, we recommend using a code-oriented LLM such as StarCoder2 or Qwen2.5-Coder via a local runner (e.g., Ollama or LM Studio). Reasonable defaults are:
  - `starcoder2:3b` (fast, works on most laptops),
  - `starcoder2:7b` or `qwen2.5-coder:3b/7b` (better code quality on machines with more memory),
  - larger models (e.g., `starcoder2:15b`, `qwen2.5-coder:14b/32b`) only if you have access to high-end lab hardware.
- **Access method:**
  - *Local runner (recommended)*: Use Ollama or LM Studio to run the model on `localhost` and expose an OpenAI-compatible endpoint (e.g., `http://localhost:11434/v1/chat/completions` for Ollama and `http://localhost:1234/v1/chat/completions` for LM Studio). In this case, the ChatUniTest plugin is configured with a dummy `apiKeys` value, the local model name in `model`, and the local URL in `url`.
  - *Cloud API*: Use a cloud model such as `gpt-4o-mini`, with a real API key purchased from the OpenAI platform. The plugin then uses the default OpenAI URL (`https://api.openai.com/v1/chat/completions`) and the actual key in `apiKeys`.
- **Key ChatUniTest parameters in `pom.xml`:**
  - `apiKeys`: must be non-empty. Use a real key for cloud models or any dummy string for local models.
  - `model`: the model identifier (e.g., `starcoder2:3b`, `qwen2.5-coder:3b`, `gpt-4o-mini`).
  - `url`: the API endpoint. Use the default OpenAI URL for cloud models or a local URL for Ollama / LM Studio.

- `phaseType` (optional): selects a specific generation scheme. For this project, the default ChatUniTest process is sufficient, while `phaseType=MUTAP` may be used in the mutation-guided experiment.
- `testNumber`, `maxRounds`, `temperature` (optional): advanced settings controlling, respectively, the number of tests, the number of repair rounds, and the diversity of generated tests. Students may experiment with these parameters if time permits.

B.5 Controlling the number of generated tests

The ChatUniTest Maven plugin exposes the parameter `testNumber`, which controls how many test cases are generated per method (default: `5`). Students may adjust this parameter either in `pom.xml`:

```
<plugin>
  <groupId>io.github.zju-aces-ise</groupId>
  <artifactId>chatunitest-maven-plugin</artifactId>
  <version>2.1.1</version>
  <configuration>
    <!-- other configuration options -->
    <testNumber>10</testNumber>
  </configuration>
</plugin>
```

or per invocation using the `-D` flag:

```
mvn chatunitest:class -DselectClass=fully.qualified.ClassName -DtestNumber=10
```

Increasing `testNumber` typically yields a more diverse test suite at the cost of additional generation time.

B.6 Supported LLM models and limitations

The ChatUniTest Maven plugin currently enforces a whitelist of model identifiers. At the time of writing, the supported values include `gpt-3.5-turbo`, `gpt-4o`, `gpt-4o-mini`, `code-llama`, and `codeqwen:v1.5-chat`. When using a local runner such as Ollama, students should:

- pull a model with a matching name (e.g., `ollama pull codeqwen:v1.5-chat`), and
- configure the plugin with `<model>codeqwen:v1.5-chat</model>` and `<url>http://localhost:11434/v1/chat/completions</url>`.

Using other model names (e.g., `starcoder2:3b`) will cause the plugin to fail with a "No Model with name ..." error, even if the local runner supports that model. Extending ChatUniTest to support additional model names requires modifying and rebuilding the plugin, which is beyond the scope of this course project.

B.7 Note on internal warnings during MUTAP runs

When running ChatUniTest with the MUTAP scheme, students may occasionally observe Java stack traces in the console, such as exceptions mentioning `PromptGenerator` or a `NullPointerException` related to a `templateName` variable. These messages indicate that an internal prompt template could not be resolved for a specific method or round of generation. However, as long as Maven reports "BUILD SUCCESS" and the plugin prints messages such as "[MUTAP] Generation finished" or "[MUTAP] Test processed", the overall generation task has completed, and the produced tests can be used as normal.

In this case, students should:

- refresh the project and inspect the `chatunitest/` directory (and, after `mvn chatunitest:copy`, the `src/test/java` directory) to see which test classes were generated;
- run the full test suite with `mvn test` or via the IDE as already mentioned in Step 4;
- proceed with coverage and mutation analysis on the available MUTAP-generated tests.

If, instead, Maven reports *"BUILD FAILURE"* and no tests are generated, students may fall back to the default ChatUniTest scheme (i.e., without `phaseType=MUTAP`) or document the issue in their project report.

B.8 Overall Workflow: Manual Tests, LLM Tests, and Mutation-based Evaluation

The intended workflow for this project can be summarized as follows:

For this course, you are expected to:

1. generate LLM-assisted tests for selected classes using ChatUniTest (with or without `phaseType=MUTAP`),
2. integrate these tests into `src/test/java`, and
3. evaluate the resulting test suites either using PIT (if available in your environment) or via a manual mutant experiment as described in the next subsection.

1. **Design and implement manual tests.** For each selected class under test, students first design and write unit tests manually in `src/test/java`, using the input-space, logic-based, and graph-based techniques introduced in the course. At this stage, all tests are written by the students (no LLM assistance).
2. **Generate LLM-assisted tests with ChatUniTest.** Next, students invoke ChatUniTest to generate additional tests using a local LLM. For a chosen class, they run:

   ```
   mvn chatunitest:class -DselectClass=fully.qualified.ClassName
   ```

   optionally with mutation-guided generation:

   ```
   mvn chatunitest:class -DselectClass=fully.qualified.ClassName -DphaseType=MUTAP
   ```

   and then copy the resulting tests into `src/test/java`:

   ```
   mvn chatunitest:copy
   ```

   Students inspect and, if necessary, correct minor issues in the generated test code (e.g., Java version settings, type casts).
3. **Run and compare test suites.** With both manual and LLM-generated tests in place, students execute the full test suite:

   ```
   mvn test
   ```

   and compare the behaviour of:

   - manual tests only, and
   - manual + LLM-generated tests

   in terms of coverage, observed failures and faults, and other relevant metrics defined in the project.
4. **Evaluate test strength via mutants (PIT or manual).** Finally, students assess the fault-detection strength of their test suites using mutation testing:

   - If PIT is available in the provided environment, a minimal configuration in `pom.xml` should be:

     ```
     <build>
       <plugins>
         <!-- Other plugins (e.g., surefire, ChatUniTest) -->

         <plugin>
           <groupId>org.pitest</groupId>
           <artifactId>pitest-maven</artifactId>
           <version>1.16.2</version> <!-- or a recent stable version -->
           <configuration>
             <!-- Code under test -->
     ```

```
        <targetClasses>
          <param>com.example.*</param>
        </targetClasses>
        <!-- Test classes -->
        <targetTests>
          <param>com.example.*</param>
        </targetTests>
        <!-- Enable JUnit 5 support if needed -->
        <junit5PluginVersion>1.2.1</junit5PluginVersion>
      </configuration>
    </plugin>

  </plugins>
</build>
```

Once configured, a baseline mutation score can be obtained by running:

```
mvn clean test
mvn org.pitest:pitest-maven:mutationCoverage
```

and comparing mutation scores for manual vs. manual + LLM tests. PIT generates an HTML report under `target/pit-reports/`, which shows, for each class, how many mutants were killed or survived by the current test suite.

- If PIT is not available, perform a manual mutant experiment in the existing or created test files by introducing small controlled changes (mutants) in the code. Manually modify the implementation by changing one small piece of logic, such as:
  - changing a comparison (e.g., `quantity >= 10` to `quantity > 10`),
  - altering a constant, or
  - removing an exception check (e.g., deleting an `if`-statement that throws an `IllegalArgumentException`).

  Then, re-run `mvn test`, and observe which test cases (manual or LLM-generated) detect the mutations and observe which test cases fail. Students should identify:
  - which mutants are detected by their manually written tests,
  - which mutants are only detected by LLM-generated tests (e.g., tests produced by ChatUniTest or MUTAP), and
  - whether MUTAP-generated tests exercise boundary conditions or exception paths that the manual tests missed.
- **Analysis.** In the report, you must describe the manual mutants you introduced, which test cases killed them, how the LLM-generated tests complemented the manually designed test suite, and whether mutation-guided generation (MUTAP) leads to stronger test suites.

This manual mutant experiment concretely illustrates the intuition behind mutation testing (a test "kills" a mutant if it fails on the mutated program) and allows students to observe the impact of LLM-assisted test generation even if PIT cannot be executed in a given environment.

B.9 FAQ: LLM-based Test Generation, Suites, and Mutation Testing

Q1: Where should I run Maven commands?

All Maven commands such as `mvn clean install`, `mvn chatunitest:class`, and `mvn test` in this project must be executed from the *project root*, i.e., the directory that contains the `pom.xml` file.

- **Command line:**

1. Open a terminal (on Windows: `cmd` or PowerShell; on macOS/Linux: a shell).
2. Change directory to the project root, e.g.:

   ```
   cd C:\Users\<your-username>\eclipse-workspace\calculator-chatunitest
   ```

3. Run the desired Maven commands, e.g.:

   ```
   mvn clean install
   mvn chatunitest:class -DselectClass=com.example.Calculator
   mvn test
   ```

- **In Eclipse (Run As → Maven build...):**
  1. Right-click the project and select *Run As → Maven build...*.
  2. Ensure the *Base directory* is the project root (where `pom.xml` is located).
  3. Enter the goal(s) in the *Goals* field, e.g.:

     ```
     clean install
     ```

     or

     ```
     chatunitest:class -DselectClass=com.example.Calculator
     ```

  4. Click *Run*.

Q2: Why do I get the error "JAR does not exist. Run mvn install first" when I run ChatUniTest?

ChatUniTest expects the compiled project JAR to be present in the `target` directory (e.g., `target/calculator-chatunitest-1.0-SNAPSHOT.jar`). If this file is missing, the plugin fails with an error such as:

```
In TestCompiler.listClassPaths:  ...JAR does not exist.  Run mvn install
first.
```

To fix this:

1. First build the project and produce the JAR:

   ```
   mvn clean install
   ```

2. Then, in a separate command (without `clean`), run ChatUniTest, for example:

   ```
   mvn chatunitest:class \
     -DselectClass=com.example.Calculator
   ```

   or, with MUTAP:

   ```
   mvn chatunitest:class \
     -DselectClass=com.example.Calculator \
     -DphaseType=MUTAP
   ```

Do *not* run `mvn clean chatunitest:class ...` in a single command, because `clean` deletes the JAR that the plugin needs.

Q3: What is the difference between `mvn chatunitest:class -DselectClass=...[-DphaseType=MUTAP]` and `mvn org.pitest:pitest-maven:mutationCoverage`? Do I need both?

Yes, they serve different purposes:

- **ChatUniTest:**

  ```
  mvn chatunitest:class \
    -DselectClass=fully.qualified.ClassName \
    [-DphaseType=MUTAP]
  ```

  invokes the ChatUniTest Maven plugin, which uses an LLM to *generate new JUnit test cases* for the specified class.

  – Without `phaseType`, it uses the default scheme.

  - With `phaseType=MUTAP`, it uses a mutation-guided scheme (MuTAP-like), where mutation testing is used as feedback inside the generation loop to request additional tests for surviving mutants.

  ChatUniTest produces test source files (initially under `chatunitest/`) which are copied into `src/test/java` via:

  ```
  mvn chatunitest:copy
  ```

  It does *not* report a mutation score.

- **PIT:**

  ```
  mvn org.pitest:pitest-maven:mutationCoverage
  ```

  invokes the PIT mutation testing plugin, which *evaluates* the current test suite (manual + LLM-generated). PIT:

  1. creates mutants of the production code,
  2. runs the tests against these mutants, and
  3. reports a mutation score and a detailed HTML report in `target/pit-reports/`.

  PIT does not generate tests; it measures test strength.

For this project, the intended workflow is:

1. Generate LLM-assisted tests with ChatUniTest (default and/or MUTAP).
2. Copy them into `src/test/java`.
3. Evaluate the resulting test suites using PIT (if available) or via a manual mutant experiment.

Q4: What does the MUTAP error "templateName is null" mean? Is it fatal?

When running ChatUniTest with `phaseType=MUTAP`, students may see console messages like:

```
An error occurred while generating the user prompt:
java.lang.NullPointerException:  Cannot invoke "String.equals(Object)"
because "templateName" is null
```

This originates from ChatUniTest's internal `PromptGenerator` when it cannot resolve a prompt template name during a MUTAP round. However, in practice:

- The exception is caught inside the plugin.
- The plugin logs the error, then continues.
- Maven still prints `[MUTAP] Generation finished`, `[MUTAP] Test processed`, and ends with `BUILD SUCCESS`.

As long as Maven reports *"BUILD SUCCESS"* and tests appear under `chatunitest/` (and then `src/test/java` after `mvn chatunitest:copy`), the MUTAP run has produced usable tests.

If, instead, Maven reports *"BUILD FAILURE"* and no tests are generated, students should fall back to the default ChatUniTest scheme (without `phaseType=MUTAP`) and document the issue in their report.

Q5: Why does `mvn test` sometimes show "Tests run: 0" even though I can run tests in Eclipse?

If Maven prints:

```
[INFO] Tests run: 0, Failures: 0, Errors: 0, Skipped: 0
```

it means the Surefire plugin did not discover any test classes. Common causes:

- LLM-generated tests are only in `chatunitest/` and have not been copied into `src/test/java`.

- Test classes are not in the correct package or folder (they must be under `src/test/java` with a matching `package` declaration, e.g. `package com.example;`).
- The JUnit and Surefire configuration is incomplete (e.g., no JUnit 5 engine or outdated Surefire version).

To fix this, students should:

1. Run:

   ```
   mvn chatunitest:copy
   ```

   to move generated tests into `src/test/java`.

2. Ensure test classes are annotated with `@Test` from JUnit 5:

   ```
   import org.junit.jupiter.api.Test;
   ```

3. Ensure `pom.xml` includes JUnit 5 and a modern Surefire plugin, for example:

   ```
   <dependency>
     <groupId>org.junit.jupiter</groupId>
     <artifactId>junit-jupiter</artifactId>
     <version>5.9.2</version>
     <scope>test</scope>
   </dependency>

   <plugin>
     <groupId>org.apache.maven.plugins</groupId>
     <artifactId>maven-surefire-plugin</artifactId>
     <version>3.1.2</version>
     <configuration>
       <useModulePath>false</useModulePath>
     </configuration>
   </plugin>
   ```

Q6: Why does `mvn -Dtest=*Calculator* test` run `Calculator_Suite` with N tests, but plain `mvn test` does not?

ChatUniTest typically generates:

- several test classes (e.g., `Calculator_add_0_0_Test`), and
- an aggregate suite class (e.g., `Calculator_Suite`) that collects and executes these tests.

By default, Surefire only includes files matching patterns like:

```
**/Test*.java, **/*Test.java, **/*Tests.java, **/*TestCase.java
```

The suite `Calculator_Suite.java` does *not* match these patterns, so plain `mvn test` may ignore it.

When running:

```
mvn -Dtest=*Calculator* test
```

Surefire is instructed to run classes whose names match `*Calculator*`, which explicitly includes `Calculator_Suite`. The JUnit platform then discovers and executes the N underlying tests referenced by the suite.

To make plain `mvn test` also run the suite, instructors can configure Surefire to include suites explicitly:

```
<includes>
  <include>**/*Test.java</include>
  <include>**/*Tests.java</include>
  <include>**/*TestCase.java</include>
  <include>**/*Suite.java</include> <!-- LLM-generated suites -->
</includes>
```

Q7: What is the warning "Ignoring test class using JUnitPlatform runner: ... Suite" and is it a problem?

When running tests, students may see:

```
WARNING: Ignoring test class using JUnitPlatform runner:
com.example.Calculator_Suite
```

This comes from JUnit's Vintage engine: the suite is annotated with `@RunWith(JUnitPlatform.class)`, which is an older JUnit 4 style bridge to the JUnit 5 platform. When running directly on the JUnit 5 platform (with JUnit 5 and a modern Surefire plugin), this annotation is redundant, and the engine logs that it is ignoring it.

This warning does *not* indicate a failure. As long as Maven reports a non-zero number of tests run and `BUILD SUCCESS`, the suite and all underlying tests have executed correctly. The warning can safely be ignored for the purposes of this project.

Q8: Do I always need a suite (e.g., `Calculator_Suite`) to get non-zero tests?

In general, JUnit tests do not require a suite; a simple class such as `FooTest` with `@Test` methods is sufficient.

However, in the current ChatUniTest setup used in this project, the generated suite (e.g., `Calculator_Suite`) often acts as the *canonical entry point* for the LLM-generated tests. The suite collects multiple generated test classes and presents them as a single test container to the JUnit platform. If the suite is not discovered (by default includes or an explicit pattern) and the individual `*Test` classes are not treated as standalone containers by the current configuration, `mvn test` may report zero tests.

For simplicity, this project assumes that:

- the generated `*_Suite` class is the main entry point for LLM-generated tests, and
- Surefire includes both `*Test.java` and `*Suite.java` patterns.

Q9: Why are tests like `Calculator_add_0_0_Test.java` not detected directly when I ignore the suite?

The Surefire `<include>` patterns (e.g., `**/*Test.java`) only control which classes are considered as *candidates* for testing; they do not guarantee that these classes will contribute tests. After the file-pattern filter, the JUnit engines (Jupiter, Vintage, etc.) perform test discovery inside each candidate class.

In this setup, the generated `Calculator_add_0_0_Test` classes are designed to be executed via the suite (e.g., `Calculator_Suite`, which references them using `@SelectClasses`). When the suite is run, the JUnit platform discovers and executes the underlying tests. When the suite is ignored, the engines may not treat those helper classes as standalone test containers under the current configuration, resulting in zero tests. Therefore, the recommended practice in this project is to run the generated suite class (via `*Suite.java` includes) rather than attempting to run each `*_Test` file directly.

Q10: Can I add a specific version of JUnit to "fix" the suite warning? Can I use both JUnit 4 and JUnit 5 in the same project?

The warning about ignoring the `JUnitPlatform` runner is not caused by an incompatible JUnit version. It is produced because the suite uses an older JUnit 4 bridge annotation (`@RunWith(JUnitPlatform.class)`) while the tests are already executed on the JUnit 5 platform.

Adding a specific JUnit 4 dependency (e.g., `junit:4.13.2`) will not remove this warning. In fact, the presence of JUnit 4/Vintage support is what enables this message.

Recommended configuration for this project:

- Use JUnit 5 as the primary test framework:

```
<dependency>
  <groupId>org.junit.jupiter</groupId>
```

```
    <artifactId>junit-jupiter</artifactId>
    <version>5.9.2</version>
    <scope>test</scope>
  </dependency>
```

- Use a modern Surefire plugin (e.g., version 3.1.2) configured for JUnit 5.

It is technically possible to include both JUnit 4 and JUnit 5 (using the Vintage engine to run JUnit 4 tests), but this is not required for the project and does not eliminate the suite warning. For the purposes of this course, students may safely ignore this warning and rely on JUnit 5 as described above.

Q11: What if PIT (`pitest-maven`) fails with environment errors?

On some machines, the PIT plugin may fail with environment-specific errors (e.g., about the "coverage generation minion" exiting with `UNKNOWN_ERROR`). If the provided lab or starter environment runs PIT successfully, students should use PIT as described in the mutation testing section of the manual.

If PIT cannot be run on a particular machine, students must still perform a *manual mutant experiment*:

1. Start from the correct implementation and run `mvn test` to confirm all tests pass.
2. Introduce a small mutation in the code (e.g., change a comparison or remove an exception check).
3. Re-run `mvn test` and observe which manual and LLM-based tests fail.
4. Discuss which mutants are killed only by LLM-generated tests and how this complements the manual test suite.

This manual mutant experiment serves as a fallback and also builds intuition about mutation testing when tools are not available.